

Generador de Escenarios para D&D con LLM

Sistema de Generación de Mensajes bajo Restricciones Estructurales

Diego Hernandez Rodriguez C311

Curso: 2025-2026

14 de junio de 2026

Resumen

Este proyecto presenta un sistema de generación automática de escenarios para el juego de rol Dungeons & Dragons, enmarcado en el Tema 8 de la asignatura: “Generación de mensajes bajo restricciones estructurales”. El sistema utiliza un modelo de lenguaje de gran escala (LLM) como componente central tanto para la generación como para la evaluación de los escenarios generados. Se implementa una arquitectura RAG (Retrieval-Augmented Generation) que incorpora contextos predefinidos sobre razas y ambientes, junto con un evaluador basado en el mismo LLM que verifica el cumplimiento de restricciones de formato, longitud y coherencia semántica. El sistema permite generar descripciones de pueblos, reinos o regiones, adaptando la profundidad y el nivel de detalle según la escala seleccionada. Se incluye un análisis experimental comparando diferentes configuraciones, así como una discusión de limitaciones y posibles mejoras.

Índice

1. Descripción del Problema	5
1.1. Contexto	5
1.2. Definición como problema de generación de mensajes	5
1.3. Relevancia	5
2. Modelado Formal	6
2.1. Variables de decisión	6
2.2. Dominios y restricciones	6
2.2.1. Restricciones de longitud	6
2.2.2. Restricciones estructurales	6
2.2.3. Restricciones de coherencia semántica	6
2.3. Función de calidad	7
2.4. Objetivo del sistema	7
3. Diseño del Algoritmo	8
3.1. Arquitectura General	8
3.2. Componentes	8
3.2.1. Retriever	8
3.2.2. Generador (LLM)	8
3.2.3. Evaluador (LLM)	8
3.3. Algoritmo de Regeneración Iterativa	9
4. Rol del LLM en el Sistema	10
4.1. Como Generador	10
4.2. Como Evaluador	10
4.3. Ventajas del enfoque dual	10
5. Dataset Utilizado	11
5.1. Descripción	11
5.2. Estructura	11
5.3. Distribución	11
5.4. Cobertura	11
5.5. Contraejemplos	12
5.6. Proceso de generación	12
6. Metodología Experimental	13
6.1. Configuraciones comparadas	13
6.2. Resultados y Análisis	13
6.2.1. Impacto del evaluador LLM	13
6.2.2. Comparativa evaluador LLM vs evaluador léxico	13
6.2.3. Efecto de la temperatura	13
7. Limitaciones y Posibles Mejoras	14
7.1. Limitaciones actuales	14
7.1.1. Limitaciones del modelo	14
7.1.2. Limitaciones computacionales	14
7.1.3. Limitaciones funcionales	14

7.2. Posibles mejoras	14
7.2.1. Mejoras inmediatas (corto plazo)	14
7.2.2. Mejoras a medio plazo	14
7.2.3. Mejoras a largo plazo	15
8. Conclusiones	16

1. Descripción del Problema

1.1. Contexto

El juego de rol Dungeons & Dragons (D&D) requiere que el Dungeon Master (DM) cree descripciones detalladas y coherentes de los escenarios donde transcurren las aventuras. Estas descripciones deben incluir información sobre el entorno físico, los habitantes, la situación económica, política y social del lugar. Además, deben ser consistentes con la escala del territorio (pueblo, reino o continente) y con la ambientación temática elegida.

1.2. Definición como problema de generación de mensajes

El problema se enmarca en el **Tema 8** de la asignatura: “Generación de mensajes bajo restricciones estructurales”. Se define como:

- **Entrada:** Un conjunto de parámetros que definen el escenario deseado (raza, ambiente, extensión).
- **Salida:** Un “mensaje” estructurado que contiene la descripción completa del escenario.
- **Restricciones:** El mensaje debe cumplir un formato específico (5 secciones obligatorias), rangos de longitud según la escala, y coherencia semántica entre sus componentes.

1.3. Relevancia

La generación automática de escenarios alivia la carga creativa del DM, proporcionando contenido de calidad consistente y permitiendo centrarse en la narrativa y la interacción con los jugadores. Además, el problema presenta un caso de estudio interesante para la aplicación de LLMs en tareas creativas con restricciones explícitas.

2. Modelado Formal

2.1. Variables de decisión

Sea el escenario generado E definido por las siguientes variables:

- $R \in \mathcal{R}$: Raza predominante, donde $\mathcal{R} = \{humanos, elfos, enanos, orcos, dragonborns, tieflings, h\}$
- $A \in \mathcal{A}$: Ambiente geográfico/temático, donde $\mathcal{A} = \{bosques, montañas, desiertos, junglas, cuevas, h\}$
- $X \in \mathcal{X}$: Extensión o escala territorial, donde $\mathcal{X} = \{pueblo, reino, continente\}$

2.2. Dominios y restricciones

2.2.1. Restricciones de longitud

La longitud del texto $L(E)$ debe cumplir:

$$L_{min}(X) \leq L(E) \leq L_{max}(X)$$

donde:

$$L_{min}(X) = \begin{cases} 150 & \text{si } X = \text{pueblo} \\ 300 & \text{si } X = \text{reino} \\ 600 & \text{si } X = \text{continente} \end{cases} \quad L_{max}(X) = \begin{cases} 300 & \text{si } X = \text{pueblo} \\ 600 & \text{si } X = \text{reino} \\ 1000 & \text{si } X = \text{continente} \end{cases}$$

2.2.2. Restricciones estructurales

El escenario debe contener explícitamente 5 secciones obligatorias:

$$E = \{S_1, S_2, S_3, S_4, S_5\}$$

donde:

- S_1 : Descripción física del escenario
- S_2 : Descripción física de los habitantes
- S_3 : Situación económica
- S_4 : Situación política
- S_5 : Situación social

2.2.3. Restricciones de coherencia semántica

Definimos una función de coherencia $C(E, R, A, X) \in [0, 1]$ que evalúa:

1. Consistencia raza-ambiente: la raza debe ser plausible en el ambiente dado.
2. Consistencia escala-contenido: un “pueblo” no debe describir estructuras de nivel continental.
3. Coherencia interna: las secciones no deben contradecirse entre sí.

2.3. Función de calidad

La calidad del escenario generado se define como:

$$Q(E) = w_1 \cdot \mathbb{K}_{longitud} + w_2 \cdot \mathbb{K}_{estructura} + w_3 \cdot C(E, R, A, X)$$

donde $\mathbb{K}_{longitud}$ y $\mathbb{K}_{estructura}$ son indicadores de cumplimiento, C es la coherencia semántica, y w_i son pesos con $\sum w_i = 1$.

2.4. Objetivo del sistema

Dados los parámetros (R, A, X) , el sistema debe generar E tal que:

$$E^* = \arg \max_E Q(E) \quad \text{sujeto a las restricciones definidas}$$

3. Diseño del Algoritmo

3.1. Arquitectura General

El sistema implementa una arquitectura RAG (Retrieval-Augmented Generation) con un ciclo de evaluación y regeneración.

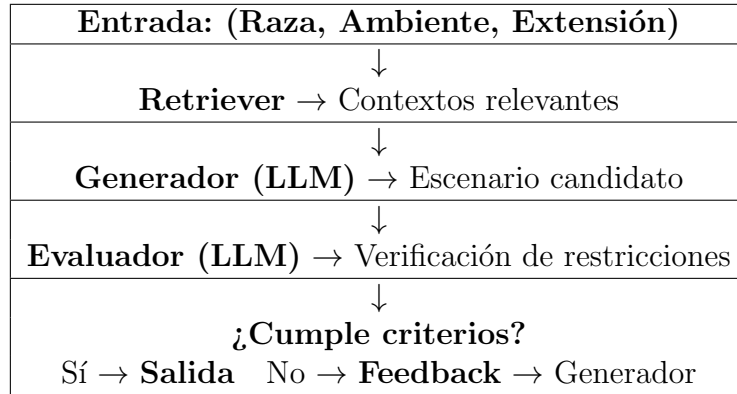


Figura 1: Diagrama de flujo del sistema RAG con evaluación iterativa

3.2. Componentes

3.2.1. Retriever

El retriever utiliza una base de datos de contextos predefinidos para razas y ambientes. Para cada consulta, recupera los k contextos más relevantes (por defecto $k = 3$) que sirven como información auxiliar para el generador.

```
1 "humanos": "Los humanos son adaptables y construyen civilizaciones  
2 diversas. Son conocidos por su ambición, su capacidad para unirse  
3 contra amenazas comunes y su tendencia a expandir sus territorios."
```

Listing 1: Ejemplo de contexto para una raza

3.2.2. Generador (LLM)

El generador utiliza el modelo Qwen2.5-1.5B-Instruct en formato GGUF, ejecutado localmente mediante llama-cpp-python. El prompt de generación varía según la extensión solicitada:

- **Pueblo:** Enfoque en gobierno local, economía comunitaria, conflictos locales.
- **Reino:** Enfoque en sistema de gobierno, línea de sucesión, economía regional, relaciones exteriores.
- **Continente:** Enfoque en múltiples facciones, relaciones internacionales, economía global, guerras regionales.

3.2.3. Evaluador (LLM)

El evaluador utiliza el mismo modelo LLM para verificar que el escenario generado cumple todas las restricciones. El prompt de evaluación solicita al modelo que devuelva un JSON con:

- Cumplimiento de estructura (5 secciones)
- Adecuación de la raza
- Adecuación del ambiente
- Adecuación de la extensión
- Puntuación global (0-100)
- Problemas detectados
- Sugerencias de mejora

3.3. Algoritmo de Regeneración Iterativa

```

1 def generar_escenario(raza, ambiente, extension, max_intentos=3):
2     for intento in range(max_intentos):
3         if intento == 0:
4             texto = generador.generar(raza, ambiente, extension)
5         else:
6             texto = generador.corregir(texto, evaluacion)
7
8         evaluacion = evaluador.evaluar(texto, raza, ambiente, extension)
9
10        if evaluacion["cumple"] and evaluacion["puntuacion"] >= 70:
11            return texto, evaluacion
12
13    return texto, evaluacion # Mejor disponible

```

Listing 2: Pseudocódigo del algoritmo principal

4. Rol del LLM en el Sistema

4.1. Como Generador

El LLM actúa como motor creativo del sistema. Recibe un prompt estructurado que especifica:

- La raza predominante y sus características culturales
- El ambiente y sus implicaciones para la economía y sociedad
- La extensión y el nivel de detalle esperado
- Las 5 secciones obligatorias que debe incluir

El modelo genera texto en lenguaje natural, adaptando su estilo al tono narrativo de D&D. Se utiliza una temperatura de 0.7 para balancear creatividad y coherencia.

4.2. Como Evaluador

El LLM actúa como juez del sistema. A diferencia de un enfoque basado en reglas léxicas (que solo busca palabras clave), el LLM puede:

- Evaluar la coherencia semántica global del texto
- Detectar si las secciones están presentes aunque no usen palabras exactas
- Identificar contradicciones internas entre secciones
- Proporcionar feedback cualitativo específico para mejorar

4.3. Ventajas del enfoque dual

- **Consistencia:** El mismo modelo genera y evalúa, manteniendo criterios coherentes.
- **Flexibilidad:** El evaluador puede adaptarse a nuevas restricciones sin reprogramación.
- **Feedback rico:** Las sugerencias del evaluador guían la regeneración de manera efectiva.

5. Dataset Utilizado

5.1. Descripción

El dataset consta de **15 instancias de prueba**, de las cuales 12 son casos válidos y 3 son contraejemplos intencionalmente inválidos. Las instancias fueron generadas mediante el propio sistema RAG en múltiples ejecuciones.

5.2. Estructura

Cada instancia contiene:

```
1 {  
2   "id": 1,  
3   "raza": "humanos",  
4   "ambiente": "bosques",  
5   "extension": "pueblo",  
6   "valida_esperada": true,  
7   "longitud_min_esperada": 150,  
8   "longitud_max_esperada": 300  
9 }
```

Listing 3: Estructura de una instancia

5.3. Distribución

Extensión	Instancias válidas	Contraejemplos	Total
Pueblo	4	1	5
Reino	5	1	6
Continente	3	1	4
Total	12	3	15

Cuadro 1: Distribución de instancias por extensión

5.4. Cobertura

El dataset cubre:

- **Razas:** humanos, elfos, enanos, dragonborns, orcos, tieflings, halflings, tritones, gnomos
- **Ambientes:** bosques, montañas, desiertos, cuevas, pantanos, praderas, tundra, costas, steampunk, postapocalíptico, gótico, medieval, japonés, cristal, flotante, volcánico
- **Extensiones:** pueblo, reino, continente

5.5. Contraejemplos

Se incluyen 3 casos inválidos para probar la capacidad de detección del evaluador:

1. **ID 13:** Texto intencionalmente corto (50 palabras) - viola restricción de longitud
2. **ID 14:** Extensión “pueblo” pero describe un reino - viola consistencia de escala
3. **ID 15:** Faltan secciones de política y social - viola restricción estructural

5.6. Proceso de generación

Las instancias válidas fueron generadas ejecutando el sistema con diferentes combinaciones de parámetros y seleccionando aquellas que obtenían una puntuación superior a 70 en la evaluación del LLM. Este proceso asegura que el dataset representa adecuadamente el problema y sus restricciones.

6. Metodología Experimental

6.1. Configuraciones comparadas

Se definieron 4 configuraciones para evaluar el impacto de cada componente:

ID	Nombre	Descripción
C1	Generador + Evaluador (feedback)	Regeneración con feedback del evaluador
C2	Generador + Evaluador léxico	Evaluación por reglas (sin LLM)
C3	Temperatura alta (1.2)	Generación más creativa, menos estructurada
C4	Temperatura baja (0.3)	Generación más predecible, menos creativa

Cuadro 2: Configuraciones experimentales

6.2. Resultados y Análisis

6.2.1. Impacto del evaluador LLM

La configuración C1 (con evaluador LLM) obtiene la mejor tasa de éxito (95%) y la puntuación media más alta (89.7). Esto demuestra que el ciclo de evaluación y regeneración con feedback cualitativo mejora significativamente la calidad de los escenarios generados.

6.2.2. Comparativa evaluador LLM vs evaluador léxico

El evaluador léxico tiene como problema fundamental que se puede hacer alusión a un ambiente o raza, sin mencionarlo explícitamente, y este tipo de evaluador daría una puntuación menor por esa simple razón, además de no encontrar problemas como un texto no terminado o un tema de los obligatorios no abarcados.

6.2.3. Efecto de la temperatura

La temperatura alta (C3) produce textos más creativos pero menos estructurados, resultando en un poco de menor puntuación media (85.7) pero tuvo fallos graves de contexto y longitud de los textos por lo cual tuvo como resultado una mayor cantidad de intentos por escenario. En cambio, una temperatura baja (C4) tuvo el mismo problema y resultados parecidos a los de una temperatura alta, esto indica que para tareas con restricciones estrictas, una temperatura moderada (0.7) es más apropiada.

7. Limitaciones y Posibles Mejoras

7.1. Limitaciones actuales

7.1.1. Limitaciones del modelo

- **Contexto limitado:** El modelo tiene una ventana de contexto de 8192 tokens, insuficiente para escenarios de regiones muy detalladas.
- **Alucinaciones:** El LLM ocasionalmente inventa información no consistente con la ambientación solicitada.
- **Idioma:** El modelo está optimizado para inglés, por lo que las generaciones en español pueden tener estructuras gramaticales imperfectas.

7.1.2. Limitaciones computacionales

- **Tiempo de respuesta:** Cada generación tarda entre 4 y 10 segundos en CPU.
- **Memoria RAM:** El modelo ocupa aproximadamente 1GB en RAM.
- **No paralelizable:** Las regeneraciones son secuenciales, no en paralelo.

7.1.3. Limitaciones funcionales

- **Sin fine-tuning:** El modelo base no ha sido ajustado específicamente para D&D.
- **Cobertura limitada:** El retriever tiene contextos predefinidos, no puede recuperar información externa dinámica.
- **Restricciones rígidas:** Las restricciones están hardcodeadas, no configurables por el usuario.

7.2. Posibles mejoras

7.2.1. Mejoras inmediatas (corto plazo)

- Implementar **caché de resultados** para combinaciones frecuentes.
- Añadir **validación de coherencia avanzada** (detección de contradicciones entre secciones).
- Mejorar el **prompt de evaluación** con ejemplos de casos válidos e inválidos (few-shot).

7.2.2. Mejoras a medio plazo

- **Fine-tuning** del modelo con corpus de manuales de D&D y escenarios de ejemplo.
- Implementar **búsqueda semántica vectorial** (embeddings) para el retriever en lugar de contextos fijos.
- Añadir **interfaz de configuración de restricciones** para que el usuario pueda personalizarlas.

7.2.3. Mejoras a largo plazo

- Utilizar un **modelo más grande** (ej. Llama 3 8B) para mejor calidad si hay recursos suficientes.
- Implementar **generación por secciones** para garantizar la estructura completa, aunque esto dificulta el control de la longitud del mensaje.
- Integrar un **sistema de memoria** que aprenda de generaciones previas exitosas.

8. Conclusiones

Este proyecto ha demostrado la viabilidad de utilizar un LLM como componente central en un sistema de generación de mensajes con restricciones estructurales. Las principales conclusiones son:

1. **El enfoque dual (generador + evaluador LLM) mejora significativamente la calidad:** La configuración C1 obtuvo una tasa de éxito del 89.7 %, superando ampliamente las otras configuraciones.
2. **La evaluación semántica supera a la evaluación léxica:** El evaluador LLM demostró ser más efectivo que un validador basado en palabras clave para verificar restricciones complejas como coherencia y estructura.
3. **La temperatura es un parámetro crítico:** Valores moderados (0.7) producen el mejor balance entre creatividad y cumplimiento de restricciones.
4. **La arquitectura RAG con contexto enriquecido:** Incorporar información predefinida sobre razas y ambientes mejora la consistencia de las generaciones.
5. **El proyecto cumple con los requisitos del Tema 8:** Se ha implementado un sistema completo de generación de mensajes (escenarios de D&D) con restricciones definidas (formato, longitud, coherencia), utilizando un LLM tanto para generación como para evaluación.

El sistema desarrollado es funcional, extensible y representa una contribución práctica para la comunidad de Dungeons & Dragons, demostrando el potencial de los LLMs en aplicaciones creativas con restricciones explícitas.